

Projet - exigences

Historique des modifications des exigences

12 mai 2026

- 6.1 - ajout "MongoDB Atlas ou autre"
- 6.3 - ajout "Simple-Schema ou autre"
- 10 - adaptation des exigences pour les tests

5 mars 2026

- première version

Création de la structure de l'interface utilisateur

À la fin de cette étape, votre projet devra respecter les critères suivants :

- **1.1 Nettoyage du projet**
 - L'application ne contient plus les styles par défaut de Vite.js.
 - Le titre de la page affiche le nom de l'application.
 - L'icône de l'application affichée dans l'onglet du navigateur a été remplacée.
- **1.2 Mise en place de l'interface utilisateur**
 - Un logo est affiché :
 - Un menu de navigation est clairement identifiable
- **1.3 Affichage sur mobile**
 - L'interface est réactive et s'adapte aux différentes tailles d'écran.
 - Le menu est caché lorsque l'écran est trop petit et reste accessible via une icône.

Navigation

À la fin de cette étape, votre projet devra respecter les critères suivants :

- **2.1 React-Router**
 - React-Router est installé et `<BrowserRouter>` englobe votre composant racine
- **2.2 Routes**
 - L'application affiche différents contenus en fonction de l'URL
- **2.3 Navigation**
 - Le menu permet de naviguer entre différentes URL de façon dynamique, sans recharger la page

Fonctionnalité

Cette exigence dépend du thème choisi pour le projet

- **3.1 Les fonctionnalités principales sont implémentées et fonctionnent**

Express.js

À la fin de cette étape, votre projet devra respecter les critères suivants :

- **4.1 express.js**
 - Vous avez créé un serveur utilisant **express.js**
 - L'application utilise **ES Modules** pour l'import des fichiers
 - Le serveur redémarre automatiquement après chaque changement grâce à **nodemon**

Connexion client/serveur

À la fin de cette étape, votre projet devra respecter les critères suivants:

- **5.1 Variables d'environnement**
 - Votre application React utilise un fichier **.env** pour stocker ses **données de configuration**, notamment l'URL du serveur
 - Vous pouvez **accéder à ces informations** grâce à **Vite.js** dans le code de l'application
- **5.2 CORS**
 - Le serveur implémente **CORS** pour éviter les erreurs de connexion du client React

MongoDB

À la fin de cette étape, votre projet devra respecter les critères suivants :

- **6.1 L'application se connecte à une base de données MongoDB sur Atlas ou autre**
 - L'URI de connexion est stockée dans un fichier .env
 - Ce fichier est ignoré par git
- **6.2 Les données pertinentes pour l'application sont stockées dans MongoDB**
 - Un événement est ajouté seulement s'il n'est pas déjà présent
- **6.3 Les collections de la base de données sont contrôlées par Simple-Schema (ou autre):**
 - Il n'est pas possible d'insérer des données corrompues dans la base de donnée collection
 - Les champs supplémentaires sont automatiquement ignorés par Simple-Schema

Authentification

À la fin de cette étape, votre projet devra respecter les critères suivants:

- **7.1 Création de compte et connexion**

- La base de données contient une collection users stockant les données de base de chaque utilisateur (nom/prénom, email, mot de passe)
- Quand l'utilisateur n'est pas authentifié, l'interface utilisateur affiche une page de login
- Sur la page de login, il est possible de créer un compte avec une adresse e-mail et un mot de passe
 - Il n'est pas possible de créer un compte avec une adresse email déjà utilisée, indépendamment de la casse
 - Après la création d'un compte, le nouvel utilisateur est automatiquement connecté
- Sur la page de login, il est possible de se connecter avec une adresse email et le mot de passe correspondant
- En cas d'échec, un message d'erreur est affiché, expliquant le problème
- En cas de succès, la page de login est remplacée automatiquement par le contenu de l'application

- **7.2 L'interface s'adapte à l'état de la connexion:**

- Si l'utilisateur n'est pas connecté, une page de login minimale s'affiche (pas de menu par exemple)
- Si l'utilisateur est connecté, une interface plus complète s'affiche, permettant de naviguer/interagir avec le contenu de l'application

- **7.3 Lorsque l'on clique sur le menu "Déconnexion", l'utilisateur est déconnecté et la page de Login s'affiche automatiquement**

- **7.4 L'authentification est implémentée avec passport.js**

- Toutes les routes qui doivent l'être sont protégées
- Il n'est plus possible d'accéder à la liste des événements sans être connecté

- **7.5 Sécurité**

- Le token d'authentification est stocké dans un cookie sécurisé (niveau de protection en fonction de l'environnement, développement ou production)
- Le token n'est pas accessible avec Javascript

GraphQL

À la fin de cette étape, votre projet devra respecter les critères suivants :

- **8.1 ApolloClient et ApolloServer**
 - ApolloClient et ApolloServer sont initialisés sur le client et le serveur respectivement
 - ApolloServer passe l'utilisateur authentifié au paramètres context de tous les resolvers
- **8.2 Queries**
 - Toutes les données sont récupérées avec GraphQL
 - Si aucun user n'est authentifié, les queries nécessitant un login retournent des réponses vides (par ex. null, [], ...)
- **8.3 Mutations protégées**
 - Les mutations GraphQL effectuent un contrôle de permissions
 - Les données liées aux utilisateurs sont assignées aux comptes des utilisateurs, le serveur utilise les données de l'utilisateur connecté pour ces assignations (pas de passage d'argument dans les mutations par ex.)
 - Les utilisateurs peuvent uniquement modifier et supprimer leurs propres données (à adapter en fonction de l'application)
- **8.4 Queries et Mutations réactives**
 - Les pages de l'application sont mises-à-jour automatiquement (pas besoin de rafraîchir la page pour voir les modifications)
 - En exécutant les queries nécessaires à nouveau
 - Ou en utilisant le cache d'Apollo

API

- **9.1 Votre back-end:**
 - Utilise au moins une API externe
 - Stocke les données récupérées dans la base de données (soit comme cache ou stockage lié à la fonctionnalité de l'application)
- **9.2 Votre front-end:**
 - Utilise et affiche les données collectées via l'API. En d'autres termes, l'API doit être pertinente pour l'application.

Tests automatisés

À la fin de cette étape, votre projet devra respecter les critères suivants:

- **10.1 Des librairies de test (par ex. Vitest et Plawright) sont installés dans les dépendances de développement de votre projet et configurés**
 - Un script pour démarrer les tests a été ajouté à package.json
- **10.2 Tests unitaires:**
 - Au moins deux tests pertinents sont implémentés
 - Tous les tests passent
- **10.3 Tests d'intégration et/ou end-to-end**
 - Au moins deux tests pertinents sont implémentés
 - Tous les tests passent